



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر
آزمایشگاه اینترنت اشیا

گزارش پروژه پنجم: مهندسی معکوس و تحلیل ثابت افزار

محمد امین حق جو ۴۰۳۱۱۰۵۸۵
امیر یوسف عبدی ۴۰۳۱۰۶۲۸۴
پارسا عدل پرور ۴۰۳۱۰۶۳۰۲

تابستان ۱۴۰۵

چکیده

در این پروژه، ما به بررسی ساختار درونی یک ثابت‌افزار و مهندسی معکوس آن می‌پردازیم و به کمک ابزار تحلیل ایستا و پویا، نحوه کار، معماری و سایر جزئیات این برنامه را استخراج می‌کنیم.

کلیدواژه‌ها: ثابت‌افزار، مهندسی معکوس، تحلیل ایستا، تحلیل پویا

فهرست مطالب

۱	مقدمه و معرفی پروژه	۱
۱-۱	تعریف مسئله	۱
۲-۱	اهمیت موضوع	۲
۳-۱	ادبیات موضوع	۲
۲	فاز اول: استخراج اطلاعات اولیه و نوع معماری	۳
۱-۲	اسکن سطحی و استخراج محتوای نهفته	۳
۲-۲	تعیین معماری هدف و نتایج این بخش	۵
۳	فاز دوم: تحلیل ایستا و پیکربندی شبیه ساز	۷
۱-۳	آماده سازی اولیه Ghidra	۷
۲-۳	تحلیل ساختار، حافظه	۷
۱-۲-۳	خواندن و شناسایی Vector Table	۷
۲-۲-۳	استخراج نقطه شروع برنامه	۹
۳-۲-۳	ساختار حافظه	۹
۳-۳	شرح سطح بالای جریان کنترل، قبل از main	۱۱
۱-۳-۳	Reset_Handler	۱۱
۲-۳-۳	cpu_init	۱۳
۳-۳-۳	kernel_init	۱۴

۱۶		thread_create	۴-۳-۳
۱۷		scheduler_start	۵-۳-۳
۱۷		تحلیل جریان کنترل نخ main	۴-۳
۱۷		(Function) main	۱-۴-۳
۱۸		FUN_08000d20	۲-۴-۳
۱۹		spi_acquire	۳-۴-۳
۱۹		(spi_transfer_byte پوشش) FUN_08000fc4	۴-۴-۳
۱۹		spi_transfer_bytes	۵-۴-۳
۲۰		جریان کلی و تعویض زمینه	۶-۴-۳
۲۱		نمایش شماتیک جریان کنترل	۷-۴-۳
۲۱		تحلیل سخت افزارهای جانبی و پروتکل های ارتباطی	۵-۳
۲۱		شناسایی Peripherals	۱-۵-۳
۲۲		روال های پیش فرض مدیریت وقفه	۲-۵-۳

۴ فاز سوم: تحلیل پویا و اعتبارسنجی یکپارچه

۲۶		تنظیمات اولیه اجرا	۱-۴
۲۹		تجميع داده ها و تنظيم نقاط توقف	۲-۴
۲۹		نقطه توقف 0x08000432 (۲)	۱-۲-۴
۲۹		نقطه توقف 0x0800043c (۳)	۲-۲-۴
۲۹		نقطه توقف 0x08000bf0 (۴)	۳-۲-۴
۳۰		نقطه توقف 0x08000d64 (۵)	۴-۲-۴
۳۰		نقطه توقف 0x08000b40 (۶)	۵-۲-۴
۳۰		نقطه توقف 0x0800062e (۷)	۶-۲-۴
۳۳		نقطه توقف 0x08000dd0 (۸)	۷-۲-۴
۳۴		نقطه توقف 0x08001094 (۹)	۸-۲-۴

۳۶		۹-۲-۴ نقطه توقف 0x0800084c (۱۰)
۳۶		۱۰-۲-۴ نقطه توقف 0x0800086a (۱۱)
۳۷		۱۱-۲-۴ نقطه توقف 0x08000884 (۱۲)
۳۷		۱۲-۲-۴ نقطه توقف 0x08000324 (۱۳)
۳۷		۱۳-۲-۴ نقطه توقف 0x08000328 (۱۴)
۳۸		۱۴-۲-۴ نقطه توقف 0x0800082c (۱۶)
۳۸		۱۵-۲-۴ نقطه توقف 0x08000838 (۱۷)
۳۹		۱۶-۲-۴ نقطه توقف 0x08000dd0 (۸)
۴۰		۱۷-۲-۴ نقطه توقف 0x08000d38 (۱۸)
۴۰		۱۸-۲-۴ نقطه توقف 0x08000d46 (۱۹)
۴۱		۱۹-۲-۴ نقطه توقف 0x08000fd8 (۲۰)
۴۱		۲۰-۲-۴ نقطه توقف 0x08000d50 (۲۱)
۴۱		۲۱-۲-۴ نقطه توقف 0x08000d54 (۲۲)

۴۳ مراجع

۴۴ آ بررسی خالی بودن خروجی binwalk

۴۶ ب آشنایی با UART و USART

۴۶		ب-۱ UART
۴۶		ب-۲ USART
۴۷		ب-۳ USART Status Register و Data Register
۴۷		ب-۳-۱ ثبات وضعیت (SR) - offset +0x00
۴۷		ب-۳-۲ ثبات داده (DR) - offset +0x04
۴۸		ب-۴ تحلیل مقایسه‌ای USART1، USART2 و USART3
۴۸		ب-۴-۱ گذرگاه فیزیکی و کارایی

ب-۴-۲ نگاهت پایه پیش فرض (بدون بازنگاشت) ۴۸

فهرست جداول

۱۰	 ساختار استخراج شده حافظه ۱-۳
۲۸	 نقاط توقف ۱-۴

فهرست تصاویر

۲۱	۱-۳ شماتیک جریان کنترل
۲۹	۱-۴ رسیدن به نقطه توقف 0x08000432
۲۹	۲-۴ رسیدن به نقطه توقف 0x0800043c
۳۰	۳-۴ رسیدن به نقطه توقف 0x08000bf0
۳۰	۴-۴ رسیدن به نقطه توقف 0x08000d64
۳۰	۵-۴ رسیدن به نقطه توقف 0x08000b40
۳۱	۶-۴ Infinite Loop
۳۲	۷-۴ Infinite Loop و پرچم‌های دست‌نخورده
۳۳	۸-۴ رسیدن به نقطه توقف 0x0800062e
۳۳	۹-۴ رسیدن به نقطه توقف 0x08000dd0
۳۶	۱۰-۴ رسیدن به نقطه توقف 0x08001094
۳۶	۱۱-۴ رسیدن به نقطه توقف 0x0800084c
۳۶	۱۲-۴ رسیدن به نقطه توقف 0x0800086a
۳۷	۱۳-۴ رسیدن به نقطه توقف 0x08000884
۳۷	۱۴-۴ رسیدن به نقطه توقف 0x08000324
۳۷	۱۵-۴ رسیدن به نقطه توقف 0x08000328
۳۸	۱۶-۴ رسیدن به نقطه توقف 0x0800082c
۳۸	۱۷-۴ رسیدن به نقطه توقف 0x08000838

۳۹	۱۸-۴ رسیدن به نقطه توقف 0x08000dd0
۴۰	۱۹-۴ رسیدن به نقطه توقف 0x08000d38
۴۰	۲۰-۴ رسیدن به نقطه توقف 0x08000d46
۴۱	۲۱-۴ رسیدن به نقطه توقف 0x08000fd8
۴۱	۲۲-۴ رسیدن به نقطه توقف 0x08000d50
۴۲	۲۳-۴ رسیدن به نقطه توقف 0x08000d54
۴۲	۲۴-۴ رسیدن به نقطه توقف نهایی در 0x08000d54 و حلقه بی‌نهایت

فصل ۱

مقدمه و معرفی پروژه

در این فصل ابتدا به بیان مسئله پروژه، اهمیت آن و اهداف پروژه می پردازیم.

۱-۱ تعریف مسئله

در بسیاری از سامانه‌های نهفته^۱ و دستگاه‌های اینترنت اشیاء^۲، کد منبع^۳ نرم‌افزار در دسترس توسعه‌دهندگان یا تحلیلگران قرار ندارد و تنها فایل باینری ثابت‌افزار^۴ روی سخت‌افزار موجود است. این محدودیت، بررسی رفتار برنامه، شناسایی اجزای مورد استفاده و درک نحوه ارتباط نرم‌افزار با سخت‌افزار را به یک چالش جدی تبدیل می‌کند. مسئله اصلی در این پروژه، مواجهه با یک فایل باینری ناشناخته به عنوان یک «جعبه سیاه» است. هدف این است که بدون داشتن سورس‌کد و با بهره‌گیری از رویکردهای مهندسی معکوس، اطلاعات حیاتی نظیر معماری پردازنده، ساختار حافظه، جریان اجرای کد^۵، توابع اصلی و نحوه مدیریت سخت‌افزارهای جانبی^۶ استخراج شده و منطق کلی سامانه به طور کامل شفاف گردد.

Embedded Systems^۱
IoT^۲
Source Code^۳
Firmware^۴
Control Flow^۵
Peripherals^۶

۲-۱ اهمیت موضوع

با گسترش روزافزون اینترنت اشیاء در زیرساخت‌های حیاتی، صنعتی و خانگی، بررسی امنیت و پایداری این تجهیزات به یک ضرورت اجتناب‌ناپذیر تبدیل شده است. تحلیل ثابت‌افزارها از طریق مهندسی معکوس، امکان کشف آسیب‌پذیری‌های امنیتی، درهای پشتی^۷ و بررسی صحت عملکرد تجهیزات را فراهم می‌سازد. علاوه بر این، در مواقعی که پشتیبانی رسمی از یک سامانه نهفته پایان یافته و یا نیاز به بررسی رفتار یک دستگاه با سخت‌افزار ناشناخته وجود دارد، مهندسی معکوس تنها راهکار عملی است. تسلط بر این فرآیند، توانمندی تیم را در تحلیل عمیق سیستم‌های نهفته ارتقا داده و پایه‌ای محکم برای پروژه‌های پیشرفته‌تر در حوزه امنیت سخت‌افزار و نرم‌افزار ایجاد می‌کند.

۳-۱ ادبیات موضوع

فرآیند مهندسی معکوس ثابت‌افزارها در ادبیات فنی معمولاً بر پایه دو رویکرد مکمل استوار است: «تحلیل ایستا»^۸ و «تحلیل پویا»^۹. در تحلیل ایستا، ساختار فایل پیش از اجرا مورد بررسی قرار می‌گیرد. این کار با استفاده از ابزارهایی نظیر Binwalk برای استخراج داده‌های اولیه و نرم‌افزارهای دیس‌اسمبلر نظیر IDA Pro یا Ghidra جهت کالبدشکافی کدهای ماشین و استخراج گراف جریان کنترل^{۱۰} انجام می‌شود. در نقطه مقابل، در تحلیل پویا، فایل باینری در یک محیط شبیه‌سازی‌شده^{۱۱} نظیر QEMU اجرا می‌گردد تا رفتار واقعی برنامه، نحوه تعامل آن با حافظه و ارتباط با درگاه‌هایی نظیر UART، SPI و GPIO در زمان اجرا پایش شود. ترکیب اصولی این دو روش، چارچوب استاندارد و جامعی است که در این پروژه نیز برای رسیدن به درک کامل از منطق ثابت‌افزار به کار گرفته خواهد شد.

Backdoors^۷

Static Analysis^۸

Dynamic Analysis^۹

CFG^{۱۰}

Emulation^{۱۱}

فصل ۲

فاز اول: استخراج اطلاعات اولیه و نوع معماری

۱-۲ اسکن سطحی و استخراج محتوای نهفته

مطابق فاز اول تعریف شده در پروپزال، ابتدا تلاش به استخراج محتوای اولیه فایل کردیم. با استفاده از دستور strings، الگوهای موجود در فایل را بررسی کردیم و خروجی آن بصورت زیر است:

```
pGpGp
K J
"P,`C
KEyS
"$hC
`ByS
JShC
SS`pG
pGpG
.F?H
9F>H
QF5H
1F2H
iF1K
1HIB
ah |
X1@Xh
bC%H%N
`#aca
@ I[ A
"@*F
2ShC
PapG
ck`+FZh
Z`ZhB
Z`Zh
JSi#
QapG
"#x0
aZiB
~_i0F9F
`0F]`
~Yi0F
+hkhC
+`k`
`ZhAi"
```

```
#XC/K
X#hC
%(Fp
)F F
 0;h#
!F0F
8`)F0F
#0F3`
`S`0F
"Fdh
lhcn
!F(F
"F(F
%#hZ
#i h
ci7F
!F(F
`#hZ
!F(F
!F(F
  #3`B
#c`#i#`
!F0F
`ci[B
a#iC
R/h+`2
j(F!j
"b`"i
`eak/`
CF:F(F!j
%(F8
(F!F
K$bcb
`` F
)F0F
pGpGpGp
SZBZA*`
#`#a
  aca
h[hR
" `c`
h[h
!`c`T`
:F0F
uhkn
)F0F
)00#0
*0#F
[F"F)F0F
;0; +N
#x.+
cx**+5
"PF!x
+0: *
*F0F
*F0F
33`#h
33`%h
IF8F
%2h0
#RFIF8F
#"FIF8F
-#mB
#iah
"aKF!F@F
h#hV
#`x#(H
!0Fbh
``ch#a
2FIF@F#i
#2FIF@F
```

```

(F;F
NMI HANDLER
MEM MANAGE HANDLER
BUS FAULT HANDLER
USAGE FAULT HANDLER
DEBUG MON HANDLER
DUMMY HANDLER
ISR stack overflowed
Stack pointer corrupted, reset to top of stack
Context before hardfault:
    r0: 0x%08lx
    r1: 0x%08lx
    r2: 0x%08lx
    r3: 0x%08lx
    r12: 0x%08lx
    lr: 0x%08lx
    pc: 0x%08lx
    psr: 0x%08lx
FSR/FAR:
CFSR: 0x%08lx
HFSR: 0x%08lx
DFSR: 0x%08lx
AFSR: 0x%08lx
BFAR: 0x%08lx
MMFAR: 0x%08lx
Misc
EXC_RET: 0x%08lx
Attempting to reconstruct state for debugging...
In GDB:
    set $pc=0x%lx
    frame 0
    bt
ISR stack overflowed by at least %d bytes.
HARD FAULT HANDLER
*** RIOT kernel panic:
*** halted.
FAILED ASSERTION.
main(): This is RIOT! (Version: 2020.01-devel-1516-g3a29d)
idle
main
#-0+
efgEFG
0123456789ABCDEF
0123456789abcdef

```

سپس دستور binwalk را اجرا کردیم و خروجی آن به ما اطلاعاتی درمورد Header نداد. درباره این موضوع، در پیوست [آ](#) بیشتر توضیح داده‌ایم.

۲-۲ تعیین معماری هدف و نتایج این بخش

دو نتیجه‌گیری از که خروجی strings کردیم:

- کد باینری داده شده، به صورت Bare-Metal استفاده نمی‌شود و بر روی RIOT-OS است.
- با معماری ARM-v7 روبرو هستیم. دلیل این برداشت، وجود رجیسترهایی از جمله r0 تا r3، lr(Link Register)، pc(Program Counter)،

(Program Status Register) psr است، که به‌طور قطعی نشان می‌دهد که با یک پردازنده از خانواده ARM روبرو هستیم. همچنین وجود رشته‌هایی مثل MEM MANAGE HANDLER، BUS FAULT HANDLER و USAGE FAULT HANDLER مختص معماری ARM Cortex-M می‌باشند. از طرفی وجود رجیسترهایی مثل CFSR، HFSR و BFAR سخت‌افزارهای پیشرفته‌ای برای دیباگ هستند که فقط در پردازنده‌های Cortex-M3 و Cortex-M4 موجود می‌باشند. [۱] پس الان مطمئن هستیم با ARM Cortex-M سروکار داریم. اما صرفاً برای اطلاع دقیق‌تر داشتن از خود پردازنده، باید بررسی کنیم که پردازنده Cortex-M3 یا Cortex-M4. برای بررسی تفاوت بین این دو، کفایت ISA این دو را بررسی کنیم چراکه Cortex-M3 برخلاف Cortex-M4 از دستور واحد FP و دستور DSP پشتیبانی نمی‌کند. [۲، ۳] بررسی این موارد در فاز دوم و در Ghidra انجام می‌گردد. با بررسی در Ghidra به این نتیجه رسیدیم که پردازنده Cortex-M3 است و همچنین با میکروکنترلرهای STM-32 روبرو هستیم.

فصل ۳

فاز دوم: تحلیل ایستا و پیکربندی شبیه‌ساز

۱-۳ آماده‌سازی اولیه Ghidra

پس از شناسایی معماری هدف ثابت‌افزار که از نوع ARM Cortex-M (32-bit Little Endian) است، و متعلق به یک میکروکنترلر STM32 می‌باشد، پس از بارگذاری فایل firmware.bin در نرم‌افزار، ابتدا معماری اخیر را انتخاب کردیم، سپس توجه داریم که ثابت‌افزار در بخش Flash حافظه قرار دارد و به این دلیل، آدرس‌ها از مقدار 0x0800_0000 شروع می‌شوند، لذا ما این موضوع را در تنظیمات Ghidra وارد کردیم.

۲-۳ تحلیل ساختار، حافظه

۱-۲-۳ خواندن و شناسایی Vector Table

در این فایل باینری، محل Vector Table در ابتدای فایل است و آدرس‌های مهم و کاربردی در آن درج گردیده‌اند. این آدرس‌ها از 0x0800_0000 تا 0x0800_00e8 قرار دارند. محتوای جدول در نرم‌افزار Ghidra ضمیمه گردیده‌است.

Initial_SP		XREF [1]:
FUN_08000d64:08000d6a (*)		
08000000 00 02 00 20 addr	DAT_20000200	
Reset_Handler		XREF [1]:
Entry Point (*)		
08000004 1d 04 00 08 addr	DAT_1000041d	
NMI_Handler		XREF [1]:
Entry Point (*)		
08000008 b9 03 00 08 addr	DAT_100003b9	

HardFault_Handler				XREF[1]:
Entry Point (*)				
0800000c	75	03	00 08 addr	DAT_10000375
MemManage_Handler				XREF[1]:
Entry Point (*)				
08000010	c9	03	00 08 addr	DAT_100003c9
BusFault_Handler				XREF[1]:
Entry Point (*)				
08000014	d9	03	00 08 addr	DAT_100003d9
UsageFault_Handler				XREF[1]:
Entry Point (*)				
08000018	e9	03	00 08 addr	DAT_100003e9
PTR_0800001c				XREF[1]:
Entry Point (*)				
0800001c	00	00	00 00 addr	00000000
PTR_08000020				XREF[1]:
Entry Point (*)				
08000020	00	00	00 00 addr	00000000
PTR_08000024				XREF[1]:
Entry Point (*)				
08000024	00	00	00 00 addr	00000000
PTR_08000028				XREF[1]:
Entry Point (*)				
08000028	00	00	00 00 addr	00000000
SVC_Handler				XREF[1]:
Entry Point (*)				
0800002c	59	03	00 08 addr	DAT_10000359
DebugMon_Handler				XREF[1]:
Entry Point (*)				
08000030	f9	03	00 08 addr	DAT_100003f9
PTR_08000034				XREF[1]:
Entry Point (*)				
08000034	00	00	00 00 addr	00000000
PendSV_Handler				XREF[1]:
Entry Point (*)				
08000038	3d	03	00 08 addr	DAT_1000033d
SysTick_Handler				XREF[1]:
Entry Point (*)				
0800003c	09	04	00 08 addr	DAT_10000409
PTR_DAT_08000040				XREF[1]:
Entry Point (*)				
08000040	ed	00	00 08 addr	DAT_100000ed
08000044	ed	00	00 08 addr	LAB_080000ec+1
08000048	ed	00	00 08 addr	LAB_080000ec+1
0800004c	ed	00	00 08 addr	LAB_080000ec+1
08000050	ed	00	00 08 addr	LAB_080000ec+1
08000054	ed	00	00 08 addr	LAB_080000ec+1
08000058	ed	00	00 08 addr	LAB_080000ec+1
0800005c	ed	00	00 08 addr	LAB_080000ec+1
08000060	ed	00	00 08 addr	LAB_080000ec+1
08000064	ed	00	00 08 addr	LAB_080000ec+1
08000068	ed	00	00 08 addr	LAB_080000ec+1
0800006c	ed	00	00 08 addr	LAB_080000ec+1
08000070	ed	00	00 08 addr	LAB_080000ec+1
08000074	ed	00	00 08 addr	LAB_080000ec+1
08000078	ed	00	00 08 addr	LAB_080000ec+1
0800007c	ed	00	00 08 addr	LAB_080000ec+1
08000080	ed	00	00 08 addr	LAB_080000ec+1
08000084	ed	00	00 08 addr	LAB_080000ec+1
08000088	ed	00	00 08 addr	LAB_080000ec+1
0800008c	ed	00	00 08 addr	LAB_080000ec+1
08000090	ed	00	00 08 addr	LAB_080000ec+1
08000094	ed	00	00 08 addr	LAB_080000ec+1
08000098	ed	00	00 08 addr	LAB_080000ec+1
0800009c	ed	00	00 08 addr	LAB_080000ec+1
080000a0	ed	00	00 08 addr	LAB_080000ec+1
080000a4	ed	00	00 08 addr	LAB_080000ec+1
080000a8	ed	00	00 08 addr	LAB_080000ec+1
080000ac	ed	00	00 08 addr	LAB_080000ec+1
080000b0	ed	00	00 08 addr	LAB_080000ec+1
080000b4	ed	00	00 08 addr	LAB_080000ec+1

080000b8	ed	00	00	08	addr	LAB_080000ec+1
080000bc	ed	00	00	08	addr	LAB_080000ec+1
080000c0	ed	00	00	08	addr	LAB_080000ec+1
080000c4	ed	00	00	08	addr	LAB_080000ec+1
080000c8	ed	00	00	08	addr	LAB_080000ec+1
080000cc	ed	00	00	08	addr	LAB_080000ec+1
080000d0	ed	00	00	08	addr	LAB_080000ec+1
080000d4	d7	07	00	08	addr	LAB_080007d6+1
080000d8	d1	07	00	08	addr	LAB_080007d0+1
080000dc	dd	07	00	08	addr	LAB_080007dc+1
080000e0	ed	00	00	08	addr	LAB_080000ec+1
080000e4	ed	00	00	08	addr	LAB_080000ec+1
080000e8	ed	00	00	08	addr	LAB_080000ec+1

۲-۲-۳ استخراج نقطه شروع برنامه

با استناد به Vector Table، آفست 0x04 نشان‌دهنده آدرس Reset_Handler است، که پس از انجام boot سخت‌افزاری یا reset سخت‌افزاری، اولین دستور واکنشی شده از حافظه، از آن آدرس است. در این ثابت‌افزار به خصوص، این آدرس برابر با 0x0800_041c است. یک نکته حائز اهمیت درباره معماری ARM Cortex-M، داشتن دو حالت ARM و Thumb است. در این معماری، پردازنده همواره در حالت Thumb است، و در نتیجه، کم‌ارزش‌ترین بیت ثابت pc برابر یک است. به همین دلیل، آدرس‌های مطلق در فایل باینری، همواره کم‌ارزش‌ترین بیت شان برابر یک است. در مورد یافتن آدرس شروع، این نکته حائز اهمیت بود، زیرا که نقطه شروع در Vector Table به 0x0800_041d درج گردیده‌بود، اما برای یافتن آدرس صحیح باید بیت کم‌ارزش آن را صفر کنیم. [۱]

۳-۲-۳ ساختار حافظه

ساختار کامل و استخراج‌شده حافظه

جدول زیر ساختار دقیق حافظه داخلی را برای هر دو بخش Flash (ROM) و System SRAM (RAM) نشان می‌دهد:

جدول ۳-۱: ساختار استخراج شده حافظه

ناحیه حافظه	محدوده آدرس	اندازه	نام بخش	توضیحات
Flash (ROM)	0x08000000 - 0x0800239f	0x23A0	.text & .rodata	کدهای اجرایی ماشین، جدول بردارهای وقفه و ثابت‌های فقط-خواندنی.
Flash (ROM)	0x080023a0 - 0x08002417	0x78	.data (Storage)	مقادیر اولیه برای متغیرهای global/static که در Flash نگهداری می‌شوند.
SRAM (RAM)	0x20000000 - 0x200001ff	0x200	System / Early Stack	فضای رزرو شده در آدرس‌های پایین RAM، که با الگوی 0xE7FEE7FE واترمارک می‌شود.
SRAM (RAM)	0x20000200 - 0x20000277	0x78	.data (Active)	متغیرهای مقداردهی شده فعال که هنگام راه‌اندازی از Flash کپی می‌شوند.
SRAM (RAM)	0x20000278 - 0x20000a97	0x820	.bss	متغیرهای global/static که با صفر مقداردهی می‌شوند.
SRAM (RAM)	0x20000a98 تا انتهای RAM	متغیر	Heap / Main Stack	فضای تخصیص داده پویا (Heap) و فریم‌های پشته (Stack) در زمان اجرا.

در تابع Reset_Handler، اگر به آدرس 0x0800_041c برویم، کدهای اسمبلی استاندارد راه‌اندازی (Boot) را می‌بینیم که این وظایف حیاتی را قبل از رسیدن به main انجام می‌دهند: تنظیم Stack Pointer. کپی کردن داده‌های مقداردهی شده (بخش .data) از حافظه Flash به RAM. صفر کردن متغیرهای مقداردهی نشده (بخش .bss) در RAM. راه‌اندازی کلاک اصلی سیستم (SystemInit).

واترمارک کردن پشته و مقداردهی اولیه RAM

08000422	mov r2, sp	; r2 = current Stack Pointer value
08000424	ldr r3, [DAT_0800046c]	; r3 = 0x20000000 (RAM start)
08000426	ldr r1, [DAT_08000470]	; r1 = 0xE7FEE7FE (Fill pattern)
08000428	cmp r3, r2	; Compare current pointer with SP
0800042a	bcc LAB_08000454	; If r3 < SP, write pattern
08000454	str.w r1, [r3], #0x4	; *r3 = 0xE7FEE7FE, r3 += 4

این بخش الگوی ۳۲-بیتی 0xe7fee7fe را کلمه به کلمه (word-by-word) درون RAM (از آدرس 0x20000000 تا آدرس فعلی اشاره‌گر پشته یا sp) می‌نویسد. این کار یک واترمارک حافظه (Memory Watermark) ایجاد می‌کند که در زمان اجرا برای محاسبه میزان مصرف پشته استفاده می‌شود.

انتقال بخش .data (از Flash به RAM)

0800042c	ldr r3, [DAT_08000474]	; r3 = 0x20000200 (RAM Dest Start)
0800042e	ldr r2, [DAT_08000478]	; r2 = 0x080023A0 (Flash Src Start)
)		
08000430	ldr r1, [DAT_0800047c]	; r1 = 0x20000278 (RAM Dest End)
08000432	cmp r3, r1	; Compare current RAM address with
End		
08000434	bcc LAB_0800045a	; If r3 < 0x20000278, keep copying
0800045a	ldr.w r0, [r2], #0x4	; r0 = *FlashAddress; FlashAddress
+= 4		
0800045e	str.w r0, [r3], #0x4	; *RAMAddress = r0; RAMAddress +=
4		

این قسمت 0x78 بایت (۱۲۰ بایت) از داده‌های مقداردهی شده اولیه (Initialized Variables) را که به صورت دائمی در آدرس 0x080023a0 حافظه Flash ذخیره شده‌اند، به ناحیه 0x20000200 تا 0x20000278 در حافظه RAM کپی می‌کند.

پاک‌سازی بخش .bss (مقداردهی با صفر)

08000436	movs r1, #0x0	; r1 = 0
08000438	ldr r3, [DAT_08000480]	; r3 = 0x20000278 (BSS RAM Start)
0800043a	ldr r2, [DAT_08000484]	; r2 = 0x20000A98 (BSS RAM End)
0800043c	cmp r3, r2	; Compare current address with End
0800043e	bcc LAB_08000464	; If r3 < 0x20000A98, keep zeroing
08000464	str.w r1, [r3], #0x4	; *RAMAddress = 0; RAMAddress += 4

این حلقه 0x820 بایت (۲۰۸۰ بایت) از حافظه RAM (از آدرس 0x20000278 تا 0x20000a98) را پیمایش کرده و در تمام خانه‌های حافظه مقدار صفر را می‌نویسد.

۳-۳ شرح سطح بالای جریان کنترل، قبل از main

جریان کنترل این ثابت‌افزار را می‌توان در تعدادی تابع مهم خلاصه نمود و تبادل کنترل میان آن‌ها را دقیق بررسی کرد. اکنون، در هر بخش، وظیفه سطح بالای هر بخش و آدرس و کدهای ماشین سطح پایین آن را توضیح و شرح می‌دهیم. برای تشخیص و شناسایی این توابع، از مخزن کد [۴] کمک گرفتیم.

Reset_Handler ۱-۳-۳

این تابع در آدرس 0x0800041c قرار دارد. این تابع در اصل نقطه شروع به کار برنامه است و پس از روشن شدن یا بازنشانی سیستم، اولین دستور از این آدرس به درون CPU واکنشی می‌شود.

این تابع، توابع دیگری را درون خود به مقاصد مختلفی فراخوانی می‌کند و کارهای مختلفی انجام می‌دهد، این‌ها عبارتند از:

- فراخوانی `board_init_pre()`
- کپی `.data` از حافظه‌ی فلش به RAM
- صفر کردن `.bss`
- فراخوانی `board_init_post()`
- فراخوانی `cpu_init()`
- فراخوانی `constructors_run()`
- فراخوانی `kernel_init()`

توابعی که مفصل تر هستند و توابع دیگری در خودشان فراخوانی می‌کنند را در بخش‌های آتی توضیح می‌دهیم.

`board_init_pre`

این تابع در آدرس `0x08000418` قرار دارد. این تابع در این پیاده‌سازی خالی است و صرفاً یک `return;` است.

`board_init_post`

این تابع در آدرس `0x0800041a` قرار دارد. این تابع در این پیاده‌سازی خالی است و صرفاً یک `return;` است.

`cpu_init`

این تابع در آدرس `0x08000bf0` قرار دارد. این بخش برای راه‌اندازی هسته و peripheral هاست. توضیح دقیق‌تر آن را در بخش مختص آن آورده‌ایم.

constructors_run

این تابع در آدرس 0x08001094 قرار دارد. این تابع، وظیفه دارد که static constructor های موجود که مربوط به بخش های C++ ثابت افزار هستند را اجرا کند.

kernel_init

این تابع در آدرس 0x0800084c قرار دارد. این بخش در نهایت فراخوانی می شود و هیچگاه return نمی کند، در عوض برنامه ریز^۱ را راه اندازی می کند.

cpu_init ۲-۳-۳

این تابع در آدرس 0x08000bf0 قرار دارد، اما یک دستور پرش به آن در آدرس 0x080007e2 موجود است و فراخوانی تابع ابتدا با پرش به 0x080007e2 انجام می شود. وظیفه این تابع، انجام امور اولیه مربوط به CPU است. روند کنترلی این تابع چنین است:

- فراخوانی cortexm_init()
- پیکربندی کلاک با فراخوانی clock_switch_to_pll_hse()
- فعال سازی کلاک های جانبی با فراخوانی rcc_clock_enable()
- فراخوانی uart_init()
- فراخوانی spi_init()

توابعی که مفصل تر هستند و توابع اساسی دیگری در خودشان فراخوانی می کنند را در بخش های آتی توضیح می دهیم.

cortexm_init

این تابع در آدرس 0x08000d64 قرار دارد. این تابع ابتدا ثبات VTOR را مقداردهی می کند و برابر با 0x08000000 قرار می دهد. این ثبات نشان گذار محل شروع Vector Table است. سپس، واحد محاسبات ممیز شناور^۲ را (در صورت وجود) فعال می کند. این امر با مقداردهی بیت های CPACR انجام

^۱Scheduler
^۲FPU

می‌شود. در نهایت، NVIC را برای تمامی interrupt ها تنظیم می‌کند به سطح اولویت صفر.

clock_switch_to_pll_hse

این تابع در آدرس 0x08000b40 قرار دارد. این تابع، HSI را فعال می‌کند و PLL را تنظیم می‌کند تا از HSE استفاده کند. و سپس کلاک سیستم را به PLL تغییر می‌دهد.

rcc_clock_enable

این تابع در آدرس 0x08000ca8 قرار دارد. این تابع، کلاک‌های پورت‌های GPIO را فعال می‌کند.

uart_init

این تابع در آدرس 0x0800062e قرار دارد. کارکرد آن، فعال‌سازی UART است. برای UART0, UART1, UART2، دو پین gpio_init_mode, gpio_init_af را ستاپ می‌کند، کلاک آن‌ها را روشن می‌کند، همچنین نرخ باود آن‌ها را تنظیم و در نهایت interrupt های پایه RX را فعال می‌کند.

spi_init

این تابع در آدرس 0x08000dd0 قرار دارد. این تابع، گذرگاه‌های SPI را فعال‌سازی می‌کند. برای هر گذرگاه، پین‌های spi_init_pins و کلاک را فعال می‌کند و مقداردهی اولیه ثبات‌هایشان را نیز انجام می‌دهد.

kernel_init ۳-۳-۳

این برچسب^۳ در آدرس 0x0800084c قرار دارد، در حین اجرای Reset Handler اجرا می‌شود و آخرین مرحله از فرآیند مقداردهی اولیه را انجام می‌دهد. برخلاف توابع قبلی، پس از اجرای این بخش، هیچ بازگشتی وجود ندارد و در عوض، برنامه‌ریز هسته را راه‌اندازی می‌کند. روند کنترلی آن به شرح زیر است:

- غیرفعال‌سازی وقفه‌ها با irq_lock() برای تضمین ایجاد اتمی ریشه‌ها.

- ایجاد ریشه بی‌کار (Idle Thread) با اولویت ۱۵ (پایین‌ترین سطح).

Label^۳

- ایجاد ریسۀ اصلی با اولویت ۷ (بالتر از ریسۀ بی‌کار).
- فراخوانی scheduler_start() برای فعال‌سازی وقفه‌ها و شروع برنامه‌ریزی.

irq_lock

این تابع در آدرس 0x08000610 قرار دارد. وظیفه‌ی آن غیرفعال‌سازی موقت وقفه‌های سراسری (global interrupts) است. این کار با دستور CPSID I و ذخیره‌سازی وضعیت قبلی PRIMASK در ثبات r0 انجام می‌شود تا بعداً بتوان توسط irq_unlock آن را بازیابی کرد. استفاده از این تابع در kernel_init تضمین می‌کند که عملیات حساس ایجاد thread ها به‌صورت صحیح انجام شود.

thread_create

همان‌طور که در بخش kernel_init مشاهده شد، این تابع دو بار فراخوانی می‌شود. آرگومان‌های ارسالی به این تابع به‌صورت زیر هستند:

- r0: آدرس پایه‌ی اشاره‌گر پشته.
- r1: اندازه‌ی پشته به بایت.
- r2: اولویت (عدد کمتر به معنای اولویت بالاتر است).
- r3: پرچم‌ها (مانند THREAD_CREATE_RUNNING).
- [sp]: اشاره‌گر به تابع نقطه‌ی ورود.
- [sp+4]: آرگومان ارسالی به ریسۀ (در اینجا صفر).

برای ریسۀ بی‌کار (idle_thread)، آدرس پشته 0x20000378، اندازه 0x100 (۲۵۶ بایت) و اولویت 15 (پایین‌ترین سطح) در نظر گرفته شده است. نقطه‌ی ورود آن آدرس 0x08000844 است که یک حلقه‌ی بی‌نهایت شامل دستور WFI (انتظار برای وقفه) می‌باشد. برای ریسۀ اصلی (main)، آدرس پشته 0x20000478، اندازه 0x600 (۱۵۳۶ بایت) و اولویت 7 در نظر گرفته شده است. نقطه‌ی ورود آن آدرس 0x0800082c است که همان تابع main کاربر بوده و پیام خوش‌آمدگویی RIOT را چاپ می‌کند.

`scheduler_start`

این تابع در آدرس 0x08000324 قرار دارد. پس از ایجاد ریشه‌ها، `kernel_init` این تابع را صدا می‌زند. کارهای انجام شده در این تابع عبارتند از:

- فراخوانی `irq_enable_global()` برای فعال‌سازی وقفه‌ها.
 - اجرای دستور `svc #1` (فراخوانی ناظر) برای تحریک استثنای `PendSV`.
 - ورود به حلقه بی‌نهایت که هرگز به آن نمی‌رسیم، زیرا `svc` باعث تعویض زمینه می‌شود.
- دستور `svc #1` باعث می‌شود که کنترل به `handler` مربوطه در حالت مدیریتی منتقل شود و در نهایت `PendSV` تحریک گردد تا اولین تعویض زمینه به ریشه‌ی اصلی (با اولویت بالاتر) انجام شود.

`thread_create` ۴-۳-۳

این تابع در آدرس 0x080008d4 قرار دارد و قلب تخصیص و راه‌اندازی ریشه‌های جدید است. جریان کنترل دقیق آن به این صورت است که ابتدا یک بلوک کنترل ریشه (TCB) از حوضچه‌ی استاتیک واقع در آدرس 0x200002cc تخصیص می‌دهد. سپس با استفاده از تابع `thread_stack_init`، پشته‌ی ریشه را برای اولین اجرا آماده می‌کند. در ادامه، وضعیت ریشه را با `sched_set_status` به `STATUS_RUNNING` تغییر می‌دهد و آن را در صف برنامه‌ریزی قرار می‌دهد. در نهایت، اگر پرچم `THREAD_CREATE_RUNNING` تنظیم شده باشد و ریشه‌ی جدید اولویت بالاتری نسبت به ریشه‌ی جاری داشته باشد، تابع `sched_switch` را فراخوانی کرده تا بلافاصله تعویض زمینه انجام شود.

`thread_stack_init`

این تابع در آدرس 0x080002bc قرار دارد. نقش آن، مقداردهی اولیه‌ی پشته‌ی ریشه به گونه‌ای است که برنامه‌ریز بتواند برای اولین بار به درستی به آن سوئیچ کند. این تابع یک قاب زمینه‌ی جعلی (`fake context`) در بالای پشته می‌سازد. به این ترتیب که ثبات‌های R4 تا R11 را با مقادیر مشخصی (مانند 0x77777777) پر می‌کند، ثبات LR (آدرس بازگشت) را برابر با آدرس تابع `thread_exit` قرار می‌دهد تا در صورت بازگشت ریشه، به درستی آزاد شود، و ثبات PC (شمارنده‌ی برنامه) را برابر با نقطه‌ی ورودی ریشه قرار می‌دهد. به این ترتیب، وقتی برنامه‌ریز برای اولین بار به این ریشه سوئیچ می‌کند، انگار که ریشه قبلاً در حال اجرا بوده و به‌طور عادی ادامه پیدا می‌کند.

`sched_set_status`

این تابع در آدرس 0x080001d0 قرار دارد. وظیفه‌ی اصلی آن مدیریت وضعیت ریشه‌ها و به‌روزرسانی بیت‌مپ اولویت‌های آماده‌به‌اجرا (runqueue bitmap) در آدرس 0x20000278 است. هنگامی که وضعیت ریشه‌ای به STATUS_RUNNING تغییر می‌کند، بیت مربوط به اولویت آن ریشه در بیت‌مپ تنظیم می‌شود تا برنامه‌ریز بداند که ریشه‌ای با آن اولویت آماده‌ی اجرا است. برعکس، وقتی ریشه‌ای بلاک می‌شود، آن بیت از بیت‌مپ پاک می‌شود تا برنامه‌ریز آن را نادیده بگیرد.

`scheduler_start` ۵-۳-۳

همان‌طور که در بخش kernel_init توضیح داده شد، این تابع در آدرس 0x08000324 قرار دارد و حلقه‌ی اصلی برنامه‌ریزی را آغاز می‌کند. اما برای تکمیل اطلاعات، زیربخش مربوط به فعال‌سازی وقفه را اینجا نیز توضیح می‌دهیم.

`irq_enable_global`

این تابع در آدرس 0x08000618 قرار دارد. با اجرای دستور CPSIE I، وقفه‌های سراسری را فعال می‌کند. این تابع درست قبل از اجرای دستور #1 svc فراخوانی می‌شود تا اطمینان حاصل شود که پس از فعال‌سازی وقفه‌ها، PendSV بتواند به‌درستی اجرا شده و تعویض زمینه را انجام دهد.

۴-۳ تحلیل جریان کنترل نخ main

در این بخش، عملکرد کلی نخ main و توابع فراخوانی‌شده توسط آن را به‌صورت سطح‌بالا بررسی می‌کنیم. چون بررسی دقیق‌تر این بخش نیز درخواست شده بود، ما کد دی‌کامپایل شده را نیز اینجا آورده‌ایم.

`(Function) main` ۱-۴-۳

این تابع ابتدا یک تابع خالی به نام Dummy4 را فراخوانی می‌کند. سپس با استفاده از printf پیام نسخه RIOT-OS را چاپ می‌کند. در نهایت تابع FUN_08000d20 را فراخوانی می‌کند که تست SPI را انجام می‌دهد. چون FUN_08000d20 هرگز باز نمی‌گردد، دستورات بعدی در main اجرا نمی‌شوند و برنامه عملاً خاتمه نمی‌یابد.

```

1 int main(void)
2 {
3     Dummy4();
4     printf("main():_This_is_RIoT!(Version: 2020.01-devel-1516-g3a29d)")
5     ;
6     FUN_08000d20();
7     return 0;
8 }

```

FUN_08000d20 ۲-۴-۳

این تابع کاراکتر 'a' را در متغیر سراسری dTX قرار می‌دهد، سپس SPI شماره 0 را مقداردهی اولیه می‌کند (spi_init). با فراخوانی spi_acquire گذرگاه SPI را تصاحب می‌کند (قفل Mutex، فعال‌سازی کلاک و تنظیم رجیسترها). سپس یک بایت ('a') را از طریق spi_transfer_byte ارسال و هم‌زمان یک بایت دریافت می‌کند و بایت دریافتی را در متغیر سراسری dRX ذخیره می‌نماید. در پایان spi_release را فراخوانی کرده و در یک حلقه بی‌نهایت باقی می‌ماند.

```

1 void FUN_08000d20(void)
2 {
3     char *puVar1;
4     char uVar2;
5
6     puVar1 = dTX_ptr;
7     *dTX_ptr = 'a';
8     spi_init(0);
9     spi_acquire(0,0xffffffff,0,2);
10    uVar2 = FUN_08000fc4(0,0xffffffff,0,*puVar1);
11    *dRX_ptr = uVar2;
12    spi_release(0);
13    do {
14        /* WARNING: Do nothing block with infinite loop */
15    } while( true );

```

۳-۴-۳ spi_acquire

این تابع با گرفتن یک Mutex از دسترسی همزمان چند thread به گذرگاه SPI جلوگیری می‌کند. سپس کلاک مربوط به SPI را فعال کرده و رجیسترهای کنترل SPI را بر اساس حالت (Mode) و فرکانس درخواستی تنظیم می‌کند. در انتها مقدار صفر را به عنوان موفقیت بازمی‌گرداند.

۴-۴-۳ FUN_08000fc4 (پوشش spi_transfer_byte)

این تابع یک بایت را از طریق spi_transfer_bytes ارسال و همزمان یک بایت دریافت می‌کند. آرگومان‌های آن شامل شماره گذرگاه، Chip Select، حالت، و داده‌ای برای ارسال است. بافرهای ارسال و دریافت روی پشته محلی ساخته می‌شوند و در نهایت بایت دریافتی بازگردانده می‌شود.

```

1 char FUN_08000fc4(void)
2 {
3     undefined4 unaff_r5;
4
5     spi_transfer_bytes();
6     return (char)((uint)unaff_r5 >> 24);
7 }
```

۵-۴-۳ spi_transfer_bytes

این تابع هسته اصلی انتقال SPI است و پارامترهای زیر را دریافت می‌کند: شماره گذرگاه (param_1)، Chip Select (param_2)، حالت ادامه (param_3)، اشاره‌گر به بافر ارسال (param_4)، اشاره‌گر به بافر دریافت (param_5) و تعداد بایت‌ها (param_6).

ابتدا بررسی می‌کند که حداقل یکی از بافرهای ارسال یا دریافت غیرتهی باشد؛ در غیر این صورت یک panic رخ می‌دهد. سپس رجیستر کنترل SPI را برای فعال‌سازی (SPE) تنظیم می‌کند. اگر Chip Select معتبر باشد (نه 0xffffffff و نه 0xffffffff00)، آن را فعال می‌کند.

سپس بر اساس اینکه کدام بافر تهی است، یکی از سه حالت زیر را اجرا می‌کند:

- فقط ارسال (بافر دریافت تهی): داده‌ها را یکی‌یکی از بافر ارسال خوانده و در رجیستر داده SPI می‌نویسد، با انتظار برای خالی شدن TXE.
 - فقط دریافت (بافر ارسال تهی): برای هر بایت، مقدار صفر را ارسال کرده و پس از آماده شدن RXNE، داده دریافتی را در بافر دریافت ذخیره می‌کند.
 - ارسال و دریافت هم‌زمان: برای هر بایت، داده را از بافر ارسال نوشته و پس از دریافت، آن را در بافر دریافت ذخیره می‌کند.
- در انتها منتظر می‌ماند تا آخرین انتقال کامل شود (TXE و RXNE و BSY بررسی می‌شوند). اگر حالت ادامه فعال نباشد و Chip Select معتبر باشد، آن را غیرفعال کرده و در صورت نیاز SPI را خاموش می‌کند. سپس بازمی‌گردد.

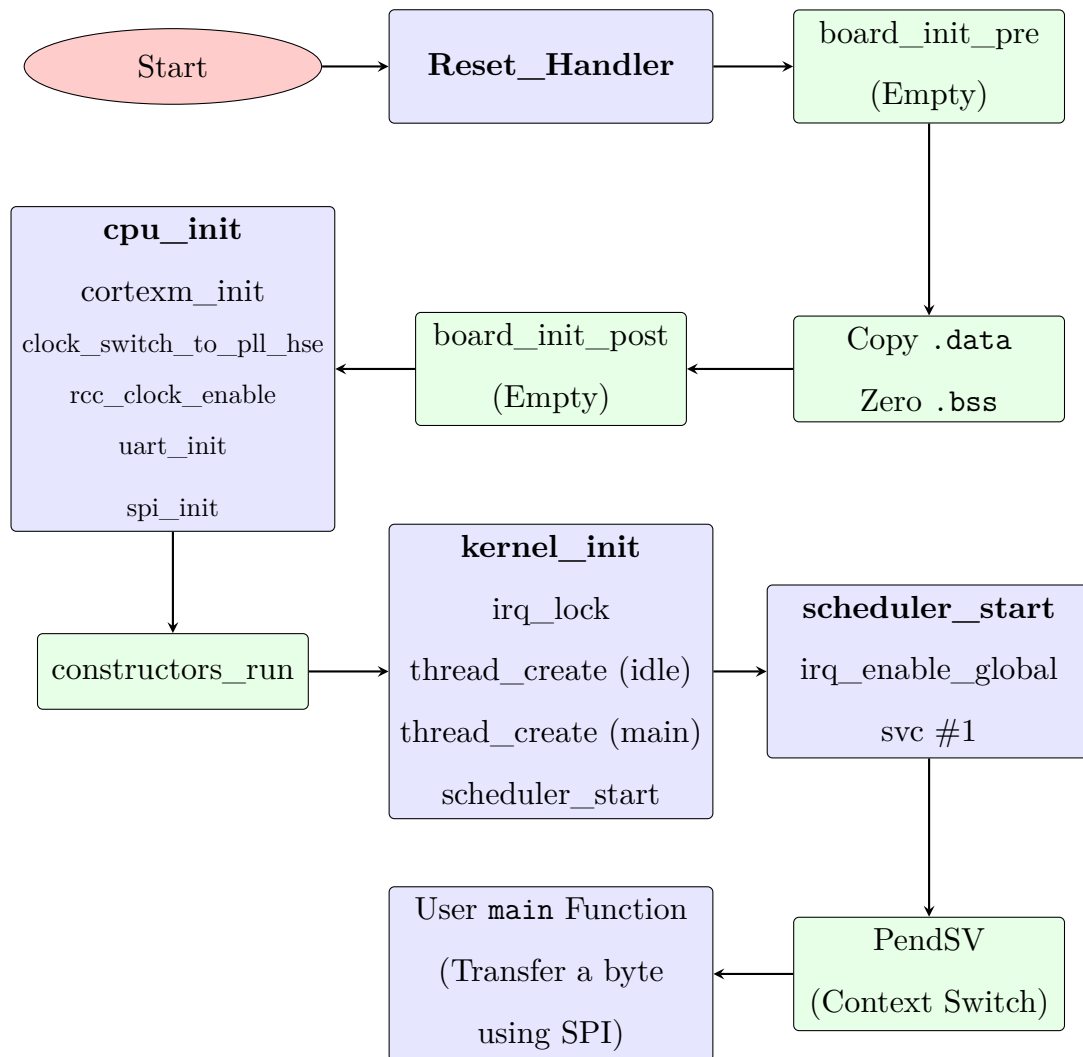
۳-۴-۶ جریان کلی و تعویض زمینه

با کنار هم قرار دادن تمام بخش‌های فوق، جریان کلی به‌صورت زیر خلاصه می‌شود:

۱. Power-On: Reset_Handler اجرا می‌شود.
 ۲. Initialization: cpu_init() سخت‌افزار و کلاک‌ها را راه‌اندازی می‌کند.
 ۳. Kernel Setup: kernel_init() ریشه‌ی idle و ریشه‌ی main را می‌سازد.
 ۴. Scheduler Start: scheduler_start() وقفه‌ها را فعال کرده و svc #1 را صدا می‌زند.
 ۵. Context Switch: دستور svc باعث تحریک PendSV می‌شود. PendSV ثبات‌های ریشه‌ی جاری (که در این مرحله همان ریشه‌ی idle یا هسته است) را ذخیره کرده و ثبات‌های ریشه‌ی main را از پشته‌ای که توسط thread_stack_init آماده شده است، بازیابی می‌کند.
 ۶. User Code: کنترل به تابع main کاربر (آدرس 0x0800082c) منتقل می‌شود، پیام خوش‌آمدگویی چاپ می‌شود و کد موجود در آنجا (ارسال و دریافت یک بایت به وسیله SPI) اجرا می‌شود.
- از این نقطه به بعد، برنامه‌ریز بر اساس وقفه‌های تایمر، IPC و بلاک‌شدن ریشه‌ها (مانند mutex_lock)، بین ریشه‌های مختلف جابه‌جا می‌شود و در صورت بیکاری همه‌ی ریشه‌ها، ریشه‌ی idle اجرا شده و پردازنده را به حالت کم‌مصرف (sleep) می‌برد.

۷-۴-۳ نمایش شماتیک جریان کنترل

جهت خلاصه‌سازی و نمایش بهتر و شهودی تر جریان کنترل، به شکل ۱-۳ مراجعه کنید.



شکل ۱-۳: شماتیک جریان کنترل

۵-۳ تحلیل سخت‌افزارهای جانبی و پروتکل‌های ارتباطی

۱-۵-۳ شناسایی Peripherals

در این گام، باید Peripheral‌هایی که توسط این برنامه مورد استفاده هستند را تشخیص دهیم.

شناسایی peripheral از طریق جدول بردارها

پردازنده Vector Table Cortex-M3 برای تعیین نقطه ورود هر استثنا و وقفه استفاده می‌کند. این جدول در ابتدای حافظه فلش (0x08000000) قرار دارد. جایگاه هر entry توسط سازنده تراشه به صورت سخت‌افزاری ثابت شده و توسط نرم‌افزار قابل تغییر نیست.

برای یک peripheral مشخص، offset در جدول بردارها به صورت زیر محاسبه می‌شود:

$$\text{Offset} = 0x40 + (\text{IRQ Number} * 4)$$

۱۶ entry نخست (افست‌های 0x00 تا 0x3c) به استثنای سیستمی (بازنشانی، HardFault و غیره) اختصاص دارند. وقفه‌های peripheral از IRQ #0 شروع می‌شوند که در افست 0x40 قرار دارد.

در حین تحلیل، نشانی‌های حافظه زیر در ناحیه فلش بررسی شدند:

Derived Handler	Stored Function Pointer	IRQ Number	Offset	Flash Address
USART1 Handler	LAB_080007d6+1	37	0xd4	0x080000d4
USART2 Handler	LAB_080007d0+1	38	0xd8	0x080000d8
USART3 Handler	LAB_080007dc+1	39	0xdc	0x080000dc

با مراجعه به راهنمای مرجع STM32F103 (RM0008)، نگاشت^۴ وقفه‌ها به صورت ثابت زیر است:

• IRQ #37: وقفه سراسری USART1

• IRQ #38: وقفه سراسری USART2

• IRQ #39: وقفه سراسری USART3

در نتیجه، برچسب‌های موجود در نشانی‌های 0x080007d0، 0x080007d6 و 0x080007dc باید به ترتیب روال‌های سرویس‌دهی وقفه^۵ برای USART2، USART1 و USART3 باشند. درباره پروتکل UART/USART، در پیوست **ب** توضیح داده‌ایم.

۳-۵-۲ روال‌های پیش‌فرض مدیریت وقفه

جدول بردارها شامل entry هایی برای تمام منابع وقفه ممکن است، از جمله آن‌هایی که به طور فعال توسط ثابت‌افزار استفاده نمی‌شوند. entry موجود در باقی نشانی‌های فلش که مربوط به وقفه‌های استفاده نشده

^۴ Mapping

^۵ ISR - Interrupt Service Routine

هستند، شامل یک اشاره گر^۶ به تابع موجود در 0x080000ec است. این تابع بلافاصله به 0x08000408 انشعاب^۷ می یابد که به عنوان یک مدیریت کننده وقفه پیش فرض^۸ عمل می کند.

این روال یک اشاره گر به رشته "DUMMY HANDLER" را بارگذاری کرده و تابع RIOT_startup را فراخوانی می کند که سیستم را مقداردهی اولیه نموده و هنگامی که از بافت وقفه^۹ فراخوانی شود، عملاً یک بازنشانی کنترل شده سیستم^{۱۰} را راه اندازی می کند. این طراحی به عنوان یک مکانیزم ایمنی عمل می کند: اگر هر وقفه غیرمنتظره رخ دهد، ثابت افزار قفل نمی کند و خطا نمی دهد. در عوض، یک پیام تشخیصی^{۱۱} خروجی می دهد و یک راه اندازی مجدد پاک^{۱۲} انجام می دهد.

لایه انتزاع نرم افزاری در برابر نگاشت سخت افزاری

یک نکته کلیدی، رابطه میان شماره IRQ و اندیس نرم افزاری است.

• ISR در 0x_080007d0 ثابت r0 را با مقدار 0x0 بارگذاری می کند.

• ISR در 0x_080007d6 ثابت r0 را با مقدار 0x1 بارگذاری می کند.

• ISR در 0x_080007dc ثابت r0 را با مقدار 0x2 بارگذاری می کند.

	LAB_080007d0+1	XREF [0,1]:	080000d8 (*)
080007d0	00 20	movs	r0,#0x0
080007d2	ff f7 59 bf	b.w	LAB_08000688
	LAB_080007d6+1	XREF [0,1]:	080000d4 (*)
080007d6	01 20	movs	r0,#0x1
080007d8	ff f7 56 bf	b.w	LAB_08000688
	LAB_080007dc+1	XREF [0,1]:	080000dc (*)
080007dc	02 20	movs	r0,#0x2
080007de	ff f7 53 bf	b.w	LAB_08000688

هر سه انشعاب به یک مدیریت کننده مشترک (LAB_08000688) می پرند که از r0 به عنوان اندیس در یک جدول توصیفگر نرم افزاری^{۱۳} در نشانی 0x08002200 استفاده می کند. این جدول، اندیس را به آدرس پایه فیزیکی (مثلاً 0x40004400 برای USART2) و بیت های فعال سازی ساعت نگاشت می کند.

بنابراین، در حالی که شماره IRQ سخت افزاری تغییرناپذیر است (37، 38، 39)، اندیس نرم افزاری (0، 1، 2) یک انتزاع دلخواه است که توسط توسعه دهنده ثابت افزار (RIOT-OS) برای راحتی انتخاب شده

^۶ pointer
^۷ branch
^۸ default and catch-all interrupt handler
^۹ interrupt context
^{۱۰} controlled system reset
^{۱۱} diagnostic message
^{۱۲} clean reboot
^{۱۳} Software Descriptor Table

است. جدول توصیفگر به عنوان مکانیزم اعتبارسنجی متقابل قطعی عمل می‌کند و تأیید می‌نماید که اندیس 0 به USART2، اندیس 1 به USART1 و اندیس 2 به USART3 اشاره دارد.

روال سرویس‌دهی وقفه مشترک UART (LAB_08000688)

روال موجود در نشانی LAB_08000688 یک مدیریت‌کننده وقفه یکپارچه است که بین هر سه peripheral USART به اشتراک گذاشته شده است. در زمان ورود، ثبات r0 شامل یک اندیس نرم‌افزاری (0، 1 یا 2) است که مشخص می‌کند کدام UART وقفه را ایجاد کرده است. مدیریت‌کننده ابتدا این اندیس را در گام entry جدول توصیفگر (20 bytes) ضرب کرده و در یک جدول توصیفگر مبتنی بر ROM واقع در نشانی 0x08002200 جستجو می‌کند و آدرس پایه فیزیکی USART متناظر را بازیابی می‌نماید. سپس ثبات وضعیت^{۱۴} peripheral را خوانده و دو پرچم خاص را که علت وقوع وقفه را نشان می‌دهند، بررسی می‌کند.

اگر پرچم انتقال کامل^{۱۵} تنظیم شده باشد، مدیریت‌کننده ثبات داده^{۱۶} را می‌خواند تا پرچم را در سخت‌افزار پاک کند، سپس یک تابع فراخوانی نرم‌افزاری^{۱۷} را که توسط درایور سطح بالاتر ارائه شده است، فراخوانی می‌کند. این تابع فراخوانی معمولاً بایت بعدی را برای ارسال بارگذاری می‌کند و تضمین می‌نماید که بافر فرستنده بدون مسدود کردن برنامه اصلی، پُر باقی بماند. پس از آن، مدیریت‌کننده پرچم بیکاری خط^{۱۸} را بررسی می‌کند. هنگامی که این پرچم تنظیم باشد، خط RX حداقل به مدت یک فریم سکوت داشته است که نشان‌دهنده پایان یک بسته ورودی است. مدیریت‌کننده دوباره ثبات داده را می‌خواند تا پرچم را پاک کرده و رویداد را تأیید کند. در نهایت، روال یک پرچم سراسری در حافظه RAM در نشانی 0x20000284 را بررسی می‌کند. اگر این پرچم غیرصفر باشد (نشان‌دهنده آن است که یک نخ^{۱۹} برای دریافت داده از این UART در حالت انتظار مسدود شده است)، مدیریت‌کننده یک استثنای PendSV را راه‌اندازی می‌کند. این کار یک تعویض زمینه^{۲۰} را به حالت تعلیق درمی‌آورد و به سیستم عامل اجازه می‌دهد تا پس از اتمام روال سرویس‌دهی وقفه، نخ منتظر را به‌طور ایمن از سر بگیرد. اگر هیچ نخ در انتظار نباشد، مدیریت‌کننده به سادگی ثبات‌ها را بازیابی کرده و به کد متوقف‌شده بازمی‌گردد.

	LAB_08000688	XREF [3]:	080007d2(j), 080007d8(j), 080007de(j)
08000688	14 23	movs	r3, #0x14
0800068a	43 43	mul	r3, r0
0800068c	0d 4a	ldr	r2, [DAT_080006c4] = 08002200h
0800068e	70 b5	push	{r4, r5, r6, lr}
08000690	d4 58	ldr	r4, [r2, r3] => DAT_08002200 = 40004400h
08000692	25 68	ldr	r5, [r4, #0x0] => DAT_40004400

Status Register^{۱۴}

TC - Transmission Complete^{۱۵}

Data Register^{۱۶}

callback^{۱۷}

Line Idle (IDLE)^{۱۸}

thread^{۱۹}

context switch^{۲۰}

```

08000694 aa 06      lsls  r2,r5,#0x1a
08000696 09 d5      bpl   LAB_080006ac
08000698 0c 23      movs  r3,#0xc
0800069a 58 43      muls  r0,r3
0800069c 0a 4a      ldr   r2,[DAT_080006c8]          = 20000350h
0800069e 61 68      ldr   r1,[r4,#0x4]=>DAT_40004404
080006a0 13 18      adds  r3,r2,r0
080006a2 1e 7a      ldrb  r6,[r3,#0x8]
080006a4 12 58      ldr   r2,[r2,r0]
080006a6 31 40      ands  r1,r6
080006a8 58 68      ldr   r0,[r3,#0x4]
080006aa 90 47      blx   r2
LAB_080006ac XREF[1]: 08000696(j)
080006ac 2b 07      lsls  r3,r5,#0x1c
080006ae 48 bf      it    mi
080006b0 63 68      ldr   r3,[r4,#0x4]=>DAT_40004404
080006b2 06 4b      ldr   r3,[DAT_080006cc]          = 20000284h
080006b4 1b 68      ldr   r3,[r3,#0x0]=>DAT_20000284
080006b6 1b b1      cbz   r3,LAB_080006c0
080006b8 bd e8 70 40 pop.w {r4,r5,r6,lr}
080006bc ff f7 36 be b.w trigger_pendsv_request_context_switch
-- Flow Override: CALL_RETURN (CALL_TERMINATOR)

```

فصل ۴

فاز سوم: تحلیل پویا و اعتبارسنجی یکپارچه

۴-۱ تنظیمات اولیه اجرا

مطبق قطعه کد زیر، GDB را به QEMU وصل می‌کنیم. در این شبیه‌سازی ما از ماشین stm32vldiscovery استفاده می‌کنیم که میکروکنترلر STM32F100 را شبیه‌سازی می‌کند. سپس مطابق جدول ۴-۱، نقاط توقف^۱ را مشخص می‌کنیم تا با انجام شبیه‌سازی، از Control flow و تحلیل Static اطمینان حاصل کنیم.

```
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0800041c in ?? ()
(gdb) set architecture armv7
The target architecture is set to "armv7".
(gdb) set arm fallback-mode thumb
(gdb) set arm force-mode thumb
(gdb) set disassemble-next-line on
(gdb) display/i $pc
1: x/i $pc
=> 0x0800041c:    push    {r3, lr}
(gdb) break *0x0800041c
Breakpoint 1 at 0x0800041c
(gdb) break *0x08000432
Breakpoint 2 at 0x08000432
(gdb) break *0x0800043c
Breakpoint 3 at 0x0800043c
(gdb) break *0x08000bf0
Breakpoint 4 at 0x08000bf0
(gdb) break *0x08000d64
Breakpoint 5 at 0x08000d64
(gdb) break *0x08000b40
Breakpoint 6 at 0x08000b40
(gdb) break *0x0800062e
Breakpoint 7 at 0x0800062e
(gdb) break *0x08000dd0
Breakpoint 8 at 0x08000dd0
```

^۱ Breakpoint

```
(gdb) break *0x08001094
Breakpoint 9 at 0x8001094
(gdb) break *0x0800084c
Breakpoint 10 at 0x800084c
(gdb) break *0x0800086a
Breakpoint 11 at 0x800086a
(gdb) break *0x08000884
Breakpoint 12 at 0x8000884
(gdb) break *0x08000324
Breakpoint 13 at 0x8000324
(gdb) break *0x08000328
Breakpoint 14 at 0x8000328
(gdb) break *0x08000844
Breakpoint 15 at 0x8000844
(gdb) break *0x0800082c
Breakpoint 16 at 0x800082c
(gdb) break *0x08000838
Breakpoint 17 at 0x8000838
(gdb) break *0x08000d38
Breakpoint 18 at 0x8000d38
(gdb) break *0x08000d46
Breakpoint 19 at 0x8000d46
(gdb) break *0x08000fd8
Breakpoint 20 at 0x8000fd8
(gdb) break *0x08000d50
Breakpoint 21 at 0x8000d50
(gdb) hbreak *0x08000d54
Hardware assisted breakpoint 22 at 0x8000d54
```

#	آدرس	تابع / مرحله	بخش تأیید شده
1	0x0800041c	Reset_Handler	Entry point
2	0x08000432	Copy .data loop start	Data initialization
3	0x0800043c	Zero .bss loop start	BSS initialization
4	0x08000bf0	cpu_init()	CPU init begins
5	0x08000d64	cortexm_init() (inside cpu_init)	VTOR / FPU / NVIC setup
6	0x08000b40	clock_switch_to_pll_hse()	Clock configuration
7	0x0800062e	uart_init()	UART initialization
8	0x08000dd0	spi_init()	SPI initialization
9	0x08001094	constructors_run()	C++ constructors / auto_init
10	0x0800084c	kernel_init()	Kernel starts
11	0x0800086a	First thread_create (idle)	Idle thread creation
12	0x08000884	Second thread_create (main)	Main thread creation
13	0x08000324	scheduler_start()	Scheduler begins
14	0x08000328	svc #1 instruction	Context switch trigger
15	0x08000844	idle_thread entry	Idle thread runs (after scheduler)
16	0x0800082c	main thread entry	Main thread runs
17	0x08000838	main → bl FUN_08000d20	Application routine entry
18	0x08000d38	Before spi_acquire()	SPI bus lock
19	0x08000d46	Before FUN_08000fc4()	spi_transfer_byte entry
20	0x08000fd8	spi_transfer_bytes()	Low-level byte transfer
21	0x08000d50	Before spi_release()	SPI bus unlock
22	0x08000d54	b.n 0x8000d54	Intentional infinite loop (application halt)

حال که نقاط را مشخص کردیم، با دستور continue ادامه می‌دهیم و مسیری که در تحلیل استاتیک مشخص کردیم را طی می‌کنیم.

۲-۴ تجمیع داده‌ها و تنظیم نقاط توقف

۱-۲-۴ نقطه توقف 0x08000432 (۲)

مطابق عکس ۱-۴ بدون هیچ مشکلی به این نقطه رسیدیم.

```
(gdb) continue
Continuing.

Breakpoint 2, 0x08000432 in ?? ()
=> 0x08000432: 428b      cmp     r3, r1
1: x/i $pc
=> 0x08000432:  cmp     r3, r1
```

شکل ۱-۴: رسیدن به نقطه توقف 0x08000432

نکته مهم این است که اگر اکنون continue بزنیم، چون داخل حلقه هستیم، مجدداً به همین نقطه توقف بر می‌گردیم. بنابراین با دستور disable 2، این نقطه توقف را غیرفعال کرده و سپس continue می‌زنیم.

۲-۲-۴ نقطه توقف 0x0800043c (۳)

مطابق عکس ۲-۴ بدون هیچ مشکلی به این نقطه رسیدیم.

```
(gdb) disable 2
(gdb) continue
Continuing.

Breakpoint 3, 0x0800043c in ?? ()
=> 0x0800043c: 4293      cmp     r3, r2
1: x/i $pc
=> 0x0800043c:  cmp     r3, r2
```

شکل ۲-۴: رسیدن به نقطه توقف 0x0800043c

مجدداً باید 3 disable را انجام دهیم تا به نقطه توقف بعدی برسیم.

۳-۲-۴ نقطه توقف 0x08000bf0 (۴)

مطابق عکس ۳-۴ بدون هیچ مشکلی به این نقطه رسیدیم. این نقطه شروع cpu_init() می‌باشد.

```
(gdb) disable 3
(gdb) continue
Continuing.

Breakpoint 4, 0x08000bf0 in ?? ()
=> 0x08000bf0: e92d 41f0      stmdb    sp!, {r4, r5, r6, r7, r8, lr}
1: x/i $pc
=> 0x8000bf0:      stmdb    sp!, {r4, r5, r6, r7, r8, lr}
```

شکل ۴-۳: رسیدن به نقطه توقف 0x08000bf0

۴-۲-۴ نقطه توقف 0x08000d64 (۵)

مطابق عکس ۴-۴ بدون هیچ مشکلی به این نقطه رسیدیم. این نقطه شروع `cortexm_init()` می باشد که داخل `cpu_init()` است.

```
(gdb) continue
Continuing.

Breakpoint 5, 0x08000d64 in ?? ()
=> 0x08000d64: 4b0c      ldr     r3, [pc, #48] @ (0x8000d98)
1: x/i $pc
=> 0x8000d64:      ldr     r3, [pc, #48] @ (0x8000d98)
```

شکل ۴-۴: رسیدن به نقطه توقف 0x08000d64

۵-۲-۴ نقطه توقف 0x08000b40 (۶)

مطابق عکس ۵-۴ بدون هیچ مشکلی به این نقطه رسیدیم. این نقطه شروع `clock_switch_to_pll_hse()` که از تنظیمات `clock` است، می باشد.

```
(gdb) continue
Continuing.

Breakpoint 6, 0x08000b40 in ?? ()
=> 0x08000b40: b538      push    {r3, r4, r5, lr}
1: x/i $pc
=> 0x8000b40:      push    {r3, r4, r5, lr}
```

شکل ۵-۴: رسیدن به نقطه توقف 0x08000b40

۶-۲-۴ نقطه توقف 0x0800062e (۷)

در مرحله قبل، پس از رسیدن به ابتدا `clock_switch_to_pll_hse()`، با اجرای دستور `continue` مطابق عکس ۶-۴ داخل یک حلقه بی نهایت گیر می کنیم و عملاً `crash` رخ می دهد.

```
(gdb) continue
Continuing.
```

شکل ۴-۶: Infinite Loop

این اتفاق به این دلیل رخ می‌دهد که در حال اجرای روتین تنظیمات کلاک (clock_switch_to_pll_hse) است. روتینی که کلاک سیستم را از نوسان‌ساز داخلی سرعت‌بالا^۲ به نوسان‌ساز خارجی سرعت‌بالا^۳ و سپس به PLL تغییر می‌دهد. این روتین پیش از ادامه کار، پرچم‌های وضعیت RCC (Reset and Clock Control) را بررسی می‌کند تا از پایدار بودن هر منبع کلاک اطمینان حاصل کند. این پرچم‌ها عبارت‌اند از:

- HSIRDY — آماده‌به‌کار بودن HSI (بیت 1 از RCC_CR در آدرس 0x40021000)
- HSERDY — آماده‌به‌کار بودن HSE (بیت 17 از RCC_CR)
- PLLRDY — آماده‌به‌کار بودن PLL (بیت 25 از RCC_CR)
- SWS — وضعیت سوئیچ کلاک سیستم (بیت‌های 2:3 از RCC_CFGR در آدرس 0x40021004)

سپس firmware منتظر می‌ماند تا پرچم مدنظر، 1 شود (یا نشان‌دهنده PLL باشد). در سخت‌افزار واقعی، این بیت‌ها پس از پایدار شدن نوسان‌ساز توسط RCC مقداردهی (set) می‌شوند. اما ماشین stm32vldiscovery در QEMU پرچم‌های آماده‌به‌کار RCC را شبیه‌سازی نمی‌کند. بنابراین این بیت‌ها برای همیشه 0 باقی می‌مانند و پردازنده (CPU) در یک حلقه بی‌نهایت می‌چرخد و منتظر پرچمی می‌ماند که هرگز تغییر نمی‌کند.

بصورت دقیق‌تر، در کد زیر، شاهد حلقه بی‌نهایت خواهیم بود:

```
1 0x8000bca:  ldr r2, [r3, #0]      ; Load RCC_CR register (address in r3 =
                                0x40021000)
2 0x8000bcc:  lsls r2, r2, #30     ; Shift left 30 → moves bit 1 (HSIRDY)
                                to bit 31
3 0x8000bce:  bpl.n 0x8000bca      ; Branch if bit 31 is clear (i.e.,
                                HSIRDY = 0)
```

Firmware در این مرحله از فرکانس‌های واقعی کلاک استفاده نمی‌کند. بلکه صرفاً بیت‌های وضعیت را بررسی می‌کند تا از آماده بودن کلاک‌ها اطمینان حاصل پیدا کند. با مقداردهی این بیت‌ها، فرایند

^۲ HSI or Internal High-Speed Oscillator
^۳ HSE or External High-Speed Oscillator

Handshake انجام شده و به کد اجازه داده می شود تا به کار خود ادامه دهد. بنابراین ما به صورت دستی در GDB مقداردهی کنیم:

```
1 set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<1) //
    HSIRDY
2 set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<17) //
    HSERDY
3 set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<25) //
    PLLRDY
4 set *(unsigned int*)0x40021004 = (*(unsigned int*)0x40021004 & ~0xc) | 0x8
    // SWS = PLL (bits 2:3 = 10)
```

اما با ایجاد این تغییرات، مجدداً مشکل حل نخواهد شد و مطابق عکس ۷-۴ باز هم با اجرای دستور continue، به مشکل خواهیم خورد. همانطور که در تصویر مشخص است، همچنان پرچم‌ها دست نخورده‌اند و 00000000 هستند.

```
(gdb) set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<1)
(gdb) set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<17)
(gdb) set *(unsigned int*)0x40021000 = *(unsigned int*)0x40021000 | (1<<25)
(gdb) set *(unsigned int*)0x40021004 = (*(unsigned int*)0x40021004 & ~0xc) | 0x8
(gdb) continue
Continuing.
x/wx 0x40021000^[[C^[[D^C
Program received signal SIGINT, Interrupt.
0x08000bcc in ?? ()
=> 0x08000bcc: 0792          lsls    r2, r2, #30
1: x/i $pc
=> 0x08000bcc: lsls    r2, r2, #30
(gdb) x/wx 0x40021000
0x40021000: 0x00000000
```

شکل ۷-۴: Infinite Loop و پرچم‌های دست نخورده

تفسیر ما از این اتفاق، عدم پیاده‌سازی قابلیت نوشتن بر روی بیت‌ها در QEMU's STM32F100 RCC peripheral می‌باشد. بنابراین مجبور هستیم بصورت قطعی (force) برنامه را جلو ببریم و از حلقه HSI، HSE، PLL و SWS عبور کنیم.

```
1 set $pc = 0x0800bd0 // End of HSI
2 set $pc = 0x0800b7a // End of HSE
3 set $pc = 0x0800b98 // End of PLL
4 set $pc = 0x0800baa // End of SWS
```

مطابق شکل ۴-۸ پس از مقداردهی دستی pc برای عبور از حلقه‌های مربوط به clock، به ابتدای

uart_init() رسیدیم.

```
(gdb) set $pc = 0x0800bd0
(gdb) continue
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x0800b76 in ?? ()
=> 0x0800b76: 0391          lsls    r1, r2, #14
1: x/i $pc
=> 0x0800b76: lsls    r1, r2, #14
(gdb) set $pc = 0x0800b7a
(gdb) continue
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x0800b94 in ?? ()
=> 0x0800b94: 0192          lsls    r2, r2, #6
1: x/i $pc
=> 0x0800b94: lsls    r2, r2, #6
(gdb) set $pc = 0x0800b98
(gdb) continue
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x0800ba2 in ?? ()
=> 0x0800ba2: f002 020c      and.w   r2, r2, #12
1: x/i $pc
=> 0x0800ba2: and.w   r2, r2, #12
(gdb) set $pc = 0x0800baa
(gdb) continue
Continuing.
Breakpoint 7, 0x080062e in ?? ()
=> 0x080062e: 2300          movs    r3, #0
1: x/i $pc
=> 0x080062e: movs    r3, #0
```

شکل ۴-۸: رسیدن به نقطه توقف 0x080062e

۴-۲-۷ نقطه توقف 0x0800dd0 (۸)

مطابق عکس ۴-۹ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی uart_init() با موفقیت انجام شده و حال در شرف اجرای spi_init() هستیم.

```
(gdb) continue
Continuing.
Breakpoint 8, 0x0800dd0 in ?? ()
=> 0x0800dd0: 2801          cmp     r0, #1
1: x/i $pc
=> 0x0800dd0: cmp     r0, #1
```

شکل ۴-۹: رسیدن به نقطه توقف 0x0800dd0

۸-۲-۴ نقطه توقف 0x08001094 (۹)

مطابق عکس ۱۰-۴ بدون هیچ مشکلی و پس از عبور مجدداً از نقطه توقف ۸ به این نقطه رسیدیم و این یعنی `spi_init()` با موفقیت انجام شده و حال در شرف اجرای `constructors_run()` هستیم.

علت اینکه ۲ بار این به نقطه می‌رسیم این است که ۲ بار این تابع توسط `cpu_init()` با مقادیر 0 و 1 فراخوانی می‌شود. بصورت دقیق‌تر در کد دی‌کامپایل شده زیر آن را نشان دادیم:

```
1 void cpu_init(void)
2
3 {
4     undefined4 *puVar1;
5     undefined4 uVar2;
6     undefined4 uVar3;
7     undefined4 *puVar4;
8     undefined4 *puVar5;
9     uint uVar6;
10    undefined4 *puVar7;
11    uint uVar8;
12
13    cortexm_init();
14    rcc_clock_enable(0,0x10000000);
15    clock_switch_to_pll_hse();
16    uVar8 = *(uint *) (DAT_08000c78 + 0x18);
17    rcc_clock_enable(1,0x1fc);
18    puVar5 = DAT_08000c8c;
19    puVar4 = DAT_08000c88;
20    uVar3 = DAT_08000c84;
21    uVar2 = DAT_08000c80;
22    puVar1 = DAT_08000c7c;
23    uVar6 = 0;
24    puVar7 = DAT_08000c7c;
25    do {
26        if (((undefined4 *) ((uint) puVar7 & 0xffffbfff) == puVar1) ||
27            ((undefined4 *) ((uint) puVar7 & 0xffffbfff) == puVar4)) || (puVar7 ==
```

```

    puVar5)) {
28  if (uVar6 == 0) {
29      *puVar7 = 0xffffffff;
30      puVar7[1] = uVar2;
31  }
32  else {
33      if ((uVar6 & 0xff) == 1) {
34          *puVar7 = uVar3;
35      }
36      else {
37          *puVar7 = 0xffffffff;
38      }
39      puVar7[1] = 0xffffffff;
40  }
41  }
42  uVar6 = uVar6 + 1;
43  puVar7 = puVar7 + 0x100;
44  } while (uVar6 != 0xc);
45  rcc_clock_enable(1,uVar8 & 0x1fc);
46  uart_init();
47  spi_init(0);
48  spi_init(1);
49  return;
50  }

```

```

Breakpoint 8, 0x08000dd0 in ?? ()
=> 0x08000dd0: 2801          cmp     r0, #1
1: x/i $pc
=> 0x08000dd0:  cmp     r0, #1
(gdb) continue
Continuing.

Breakpoint 8, 0x08000dd0 in ?? ()
=> 0x08000dd0: 2801          cmp     r0, #1
1: x/i $pc
=> 0x08000dd0:  cmp     r0, #1
(gdb) continue
Continuing.

Breakpoint 9, 0x08001094 in ?? ()
=> 0x08001094: b570          push    {r4, r5, r6, lr}
1: x/i $pc
=> 0x08001094:  push    {r4, r5, r6, lr}

```

شکل ۴-۱۰: رسیدن به نقطه توقف 0x08001094

۴-۲-۹ نقطه توقف 0x0800084c (۱۰)

مطابق عکس ۴-۱۱ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی `constructors_run()` با موفقیت انجام شده و حال در شرف اجرای `kernel_init()` هستیم.

```

(gdb) continue
Continuing.

Breakpoint 10, 0x0800084c in ?? ()
=> 0x0800084c: b51f          push    {r0, r1, r2, r3, r4, lr}
1: x/i $pc
=> 0x0800084c:  push    {r0, r1, r2, r3, r4, lr}

```

شکل ۴-۱۱: رسیدن به نقطه توقف 0x0800084c

۴-۲-۱۰ نقطه توقف 0x0800086a (۱۱)

مطابق عکس ۴-۱۲ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی `kernel_init()` با موفقیت انجام شده و اکنون به ادامه کار در `Reset_Handler` می پردازیم و اکنون در شرف ساخت رشته (Thread) اول که Idle می باشد، هستیم.

```

(gdb) continue
Continuing.

Breakpoint 11, 0x0800086a in ?? ()
=> 0x0800086a: f000 f833     bl      0x80008d4
1: x/i $pc
=> 0x0800086a:  bl      0x80008d4

```

شکل ۴-۱۲: رسیدن به نقطه توقف 0x0800086a

۱۱-۲-۴ نقطه توقف 0x08000884 (۱۲)

مطابق عکس ۱۳-۴ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی رشته Idle ساخته شده و قدم بعد که پیش روی داریم، ساخت رشته دوم که رشته Main می باشد.

```
(gdb) continue
Continuing.

Breakpoint 12, 0x08000884 in ?? ()
=> 0x08000884: f000 f826      bl      0x80008d4
1: x/i $pc
=> 0x8000884:      bl      0x80008d4
```

شکل ۱۳-۴: رسیدن به نقطه توقف 0x08000884

۱۲-۲-۴ نقطه توقف 0x08000324 (۱۳)

مطابق عکس ۱۴-۴ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی هر دو رشته Idle و Main ساخته شده اند. قدم بعد فراخوانی scheduler_start() است.

```
(gdb) continue
Continuing.

Breakpoint 13, 0x08000324 in ?? ()
=> 0x08000324: f000 f978      bl      0x8000618
1: x/i $pc
=> 0x8000324:      bl      0x8000618
```

شکل ۱۴-۴: رسیدن به نقطه توقف 0x08000324

۱۳-۲-۴ نقطه توقف 0x08000328 (۱۴)

مطابق عکس ۱۵-۴ بدون هیچ مشکلی به این نقطه رسیدیم و این یعنی scheduler_start() با موفقیت اجرا شده.

```
(gdb) continue
Continuing.

Breakpoint 14, 0x08000328 in ?? ()
=> 0x08000328: df01          svc      1
1: x/i $pc
=> 0x8000328:      svc      1
```

شکل ۱۵-۴: رسیدن به نقطه توقف 0x08000328

۱۴-۲-۴ نقطه توقف 0x0800082c (۱۶)

مطابق عکس ۱۶-۴ بدون هیچ مشکلی به این نقطه رسیدیم و رشته Main شروع به اجرا می‌کند. توجه داشته باشیم که رشته Idle اولویت کمتری نسبت به رشته Main داشت بنابراین ابتدا وارد Main می‌شویم.

```
(gdb) continue
Continuing.

Breakpoint 16, 0x0800082c in ?? ()
=> 0x0800082c: b508          push    {r3, lr}
1: x/i $pc
=> 0x800082c: push    {r3, lr}
```

شکل ۱۶-۴: رسیدن به نقطه توقف 0x0800082c

توجه شود که بدنه قسمت Main مطابق زیر است:

```
1  undefined4 main(void)
2  {
3      Dummy4();
4      printf(DAT_08000840);
5      FUN_08000d20();
6      return 0;
7  }
```

نقطه توقف بعدی را دقیقاً قبل از فراخوانی FUN_08000d20 قرار می‌دهیم تا با خیال راحت وارد آن شویم و در مراحل جلوتر، به بررسی به پردازیم. (تابع printf که مشاهده می‌شود، دقیقاً همان جایی است که باعث می‌شود عبارت "main(): This is RIOT! (Version: 2020.01-devel-1516-g3a29d)" را در خروجی strings مشاهده کردیم.)

۱۵-۲-۴ نقطه توقف 0x08000838 (۱۷)

مطابق عکس ۱۷-۴ بدون هیچ مشکلی به این نقطه رسیدیم و در قدم بعدی، وارد خود FUN_08000d20 می‌شویم.

```
(gdb) continue
Continuing.

Breakpoint 17, 0x08000838 in ?? ()
=> 0x08000838: f000 fa72          bl      0x8000d20
1: x/i $pc
=> 0x8000838: bl      0x8000d20
```

شکل ۱۷-۴: رسیدن به نقطه توقف 0x08000838

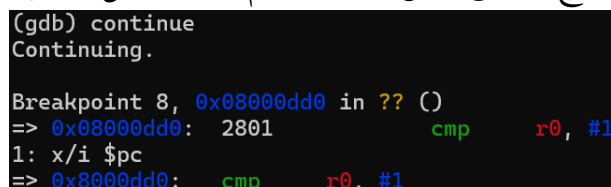
توجه شود که بدنه FUN_08000d20 مطابق زیر است:

```
1 void FUN_08000d20(void)
2
3 {
4     undefined1 *puVar1;
5     char uVar2;
6
7     puVar1 = dTX_ptr;
8     *dTX_ptr = 0x61;
9     spi_init(0);
10    spi_acquire(0,0xffffffff,0,2);
11    uVar2 = FUN_08000fc4(0,0xffffffff,0,*puVar1);
12    *dRX_ptr = uVar2;
13    spi_release(0);
14    do {
15        /* WARNING: Do nothing block with infinite loop */
16    } while( true );
17 }
```

ما اولین نقطه توقف بعدی مان را دقیقا قبل از اجرای spi_init(0) قرار دادیم.

۱۶-۲-۴ نقطه توقف 0x08000dd0 (۸)

مطابق عکس ۱۸-۴ بدون هیچ مشکلی به این نقطه رسیدیم که دقیقا قبل از اجرای spi_init(0) می باشد.



```
(gdb) continue
Continuing.

Breakpoint 8, 0x08000dd0 in ?? ()
=> 0x08000dd0: 2801          cmp     r0, #1
1: x/i $pc
=> 0x8000dd0:      cmp     r0, #1
```

شکل ۱۸-۴: رسیدن به نقطه توقف 0x08000dd0

در قدم بعدی، در تابع FUN_08000d20 جلوتر می رویم و نقطه توقف بعدی را دقیقا پس از اجرای spi_init(0) و دقیقا قبل از اجرای spi_acquire قرار می دهیم.

۱۷-۲-۴ نقطه توقف 0x08000d38 (۱۸)

مطابق عکس ۱۹-۴ بدون هیچ مشکلی به این نقطه رسیدیم که دقیقاً قبل از اجرای spi_acquire می‌باشد.

```
(gdb) continue
Continuing.

Breakpoint 18, 0x08000d38 in ?? ()
=> 0x08000d38: f000 f876      bl      0x8000e28
1: x/i $pc
=> 0x8000d38:      bl      0x8000e28
```

شکل ۱۹-۴: رسیدن به نقطه توقف 0x08000d38

در قدم بعدی، در تابع FUN_08000d20 جلوتر می‌رویم و نقطه توقف بعدی را دقیقاً پس از اجرای spi_acquire و دقیقاً قبل از اجرای FUN_08000fc4 قرار می‌دهیم (که یک وارpper برای spi_transfer_byte() می‌باشد).

۱۸-۲-۴ نقطه توقف 0x08000d46 (۱۹)

مطابق عکس ۲۰-۴ بدون هیچ مشکلی به این نقطه رسیدیم که بعد از اجرای spi_acquire و قبل از فراخوانی FUN_08000fc4 می‌باشد.

```
(gdb) continue
Continuing.

Breakpoint 19, 0x08000d46 in ?? ()
=> 0x08000d46: f000 f93d      bl      0x8000fc4
1: x/i $pc
=> 0x8000d46:      bl      0x8000fc4
```

شکل ۲۰-۴: رسیدن به نقطه توقف 0x08000d46

بدنه تابع FUN_08000fc4 مطابق زیر است:

```
1  undefined1 FUN_08000fc4(void)
2
3  {
4      undefined4 unaff_r5;
5
6      spi_transfer_bytes();
7      return (char)((uint)unaff_r5 >> 0x18);
8  }
```

در قدم بعدی، در تابع FUN_08000fc4 جلوتر می‌رویم و نقطه توقف بعدی را دقیقاً قبل از اجرای

spi_transfer_bytes() قرار می‌دهیم.

۱۹-۲-۴ نقطه‌توقف 0x08000fd8 (۲۰)

مطابق عکس ۲۱-۴ بدون هیچ مشکلی به این نقطه رسیدیم که قبل از اجرای spi_transfer_bytes() می‌باشد.

```
(gdb) continue
Continuing.

Breakpoint 20, 0x08000fd8 in ?? ()
=> 0x08000fd8: f7ff ff84      bl      0x8000ee4
1: x/i $pc
=> 0x08000fd8:      bl      0x8000ee4
```

شکل ۲۱-۴: رسیدن به نقطه‌توقف 0x08000fd8

در قدم بعدی به تابع FUN_08000d20 باز می‌گردیم و نقطه توقف بعدی را دقیقاً قبل از اجرای spi_release() قرار می‌دهیم.

۲۰-۲-۴ نقطه‌توقف 0x08000d50 (۲۱)

مطابق عکس ۲۲-۴ بدون هیچ مشکلی به این نقطه رسیدیم که قبل از اجرای spi_release() می‌باشد.

```
(gdb) continue
Continuing.

Breakpoint 21, 0x08000d50 in ?? ()
=> 0x08000d50: f000 f8a8      bl      0x8000ea4
1: x/i $pc
=> 0x08000d50:      bl      0x8000ea4
```

شکل ۲۲-۴: رسیدن به نقطه‌توقف 0x08000d50

در قدم بعد، پس از اجرای spi_release()، وارد حلقه بی‌نهایت برنامه خواهیم شد. بنابراین آخرین نقطه‌توقف را دقیقاً بعد از اجرای spi_release() و قبل از شروع حلقه قرار می‌دهیم تا از اجرای spi_release() اطمینان حاصل پیدا کنیم.

۲۱-۲-۴ نقطه‌توقف 0x08000d54 (۲۲)

مطابق عکس ۲۳-۴ بدون هیچ مشکلی به این نقطه رسیدیم که بعد از اجرای spi_release() می‌باشد.

```
(gdb) continue
Continuing.

Breakpoint 22, 0x08000d54 in ?? ()
=> 0x08000d54: e7fe b.n 0x8000d54
1: x/i $pc
=> 0x8000d54: b.n 0x8000d54
```

شکل ۴-۲۳: رسیدن به نقطه توقف 0x08000d54

مطابق عکس ۴-۲۴ از این مرحله به بعد وارد حلقه بی‌نهایت برنامه خواهیم شد و با اجرای دستور continue پیشروی دیگر نخواهیم داشت و در حالت Continuing. باقی خواهیم ماند. دلیل این اتفاق دقیقا همان while(1) بود که در انتهای FUN_08000d20 قرار داشت. کد این قسمت در Ghidra نیز به این صورت است:

1	08000d50	00	f0	a8	b1	spi_release	
						undefined spi_release()	
2				f8			
3					LAB_08000d54		XREF [1] :
					08000d54 (j)		
4	08000d54	fe	e7		b	LAB_08000d54	
5	08000d56	00	bf		nop		

```
(gdb) continue
Continuing.

Breakpoint 22, 0x08000d54 in ?? ()
=> 0x08000d54: e7fe b.n 0x8000d54
1: x/i $pc
=> 0x8000d54: b.n 0x8000d54
(gdb) disable 22
(gdb) continue
Continuing.
```

شکل ۴-۲۴: رسیدن به نقطه توقف نهایی در 0x08000d54 و حلقه بی‌نهایت

در نهایت، با استفاده از تحلیل پویا، به صحت تحلیل ایستا رسیدیم.

مراجع

- [1] ARM. Arm cortex-m3 processor datasheet. <https://support.arm.com/documentation/102831/latest>, 2020.
- [2] STMicroelectronics. Datasheet - stm32f405xx stm32f407xx. <https://www.st.com/resource/en/datasheet/dm00037051.pdf>, 2026.
- [3] STMicroelectronics. Datasheet - stm32f103x8 stm32f103xb. <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>, 2025.
- [4] RIOT-OS Community. Riot. git.riot-os. <https://git.riot-os.org/murdock/RIOT/src/commit/513b20ffd328934c58af169e2bce0c0a01eddee2>, 2016.

پیوست آ

بررسی خالی بودن خروجی binwalk

ابزار binwalk یکی از کاربردی‌ترین نرم‌افزارها در مهندسی معکوس برای تحلیل ساختار فایل‌ها است که بر اساس جستجوی امضاها (Magic Bytes/Signatures) کار می‌کند. در تحلیل فایل firmware.bin، خروجی این ابزار کاملاً خالی بود. این مسئله نشان‌دهنده خطای تحلیلی نیست، بلکه به روشنی ماهیت و ساختار این فایل را مشخص می‌کند. دلایل اصلی این موضوع عبارت‌اند از:

۱. **ماهیت فایل Raw Binary در برابر ELF:** فایل‌های اجرایی در سیستم‌عامل‌ها معمولاً دارای ساختارهای استاندارد مانند ELF (Executable and Linkable Format) یا PE هستند که هدرهای مشخصی دارند. ابزار binwalk این هدرها را به راحتی شناسایی می‌کند. اما فایل firmware.bin در میکروکنترلرها (مانند سری STM32) یک کپی مستقیم و بیت‌به‌بیت از حافظه Flash است. این فایل فاقد هرگونه هدر، متادیتا یا ساختار فایل سیستم بوده و مستقیماً با جدول بردارهای وقفه (Vector Table) و دستورات ماشین ARM آغاز می‌شود که هیچ امضای استاندارد در دیتابیس binwalk ندارند.

۲. **تفاوت مفهوم Stripped Binary با Raw Binary:** ممکن است این تصور ایجاد شود که فایل از نوع Stripped ELF است، اما این برداشت دقیق نیست. فرآیند Stripping تنها نمادهای اشکال‌زدایی (Debug Symbols) و نام توابع را از فایل ELF حذف می‌کند، اما هدر اصلی ELF و ساختار بخش‌ها (Sections) دست‌نخورده باقی می‌ماند و binwalk همچنان آن را به عنوان فایل ELF تشخیص می‌دهد.

در مقابل، فایل‌های bin. میکروکنترلی با استفاده از ابزارهایی نظیر objcopy (و با آرگومان -O-binary) تولید می‌شوند. این دستور تمامی هدرها و ساختار ELF را دور انداخته و تنها محتوای

خالص باینری (بخش‌های text و data) را استخراج می‌کند. بنابراین فایل مورد بررسی فراتر از یک فایل Stripped بوده و در واقع یک تصویر حافظه (Memory Image) کاملاً عریان است.

۳. **عدم وجود سیستم‌فایل (File System) و داده‌های فشرده:** ابزار binwalk در شناسایی سیستم‌فایل‌های تعبیه‌شده (مانند SquashFS و JFFS2) یا آرشیوهای فشرده (مانند gzip، LZMA و tar) بسیار قدرتمند است. این موارد معمولاً در Firmware روترها یا تجهیزات دارای سیستم‌عامل لینوکس تعبیه‌شده (Embedded Linux) یافت می‌شوند. از آنجا که سیستم مورد بررسی یک برنامه Bare-Metal (یا مبتنی بر یک RTOS ساده) است، کل نرم‌افزار به صورت یکپارچه کامپایل شده و هیچ سیستم‌فایل یا فایل فشرده‌ای درون آن وجود ندارد.

علت اینکه ابزار strings رشته‌های متنی را از فایل استخراج می‌کند اما binwalk هیچ خروجی ارائه نمی‌دهد این است که strings بدون توجه به ساختار یا هدر فایل، تنها به دنبال توالی کاراکترهای قابل چاپ (ASCII) می‌گردد. در مقابل، binwalk نیازمند ساختارها و امضاها (شناخته‌شده فایل‌ها (مانند ZIP، JPEG، ELF و غیره) است.

پیوست ب

آشنایی با UART و USART

ب-۱ UART

UART^۱ یک peripheral سخت‌افزاری است که داده‌ها را بین قالب موازی (بایت‌گونه) و سریال (بیت‌به‌بیت) ترجمه می‌کند. این peripheral به صورت ناهم‌زمان^۲ کار می‌کند، به این معنا که هیچ سیگنال کلاک مشترکی همراه با داده ارسال نمی‌شود. در عوض، فرستنده و گیرنده باید از پیش روی نرخ^۳ باود^۴ ثابتی (بیت بر ثانیه) توافق کنند. ارتباط از طریق دو خط اصلی انجام می‌شود:

• TX (Transmit): داده را از دستگاه خارج می‌کند.

• RX (Receive): داده را به داخل دستگاه می‌آورد.

ب-۲ USART

USART^۴ با افزودن قابلیت‌های هم‌زمان، UART را گسترش می‌دهد. در حالت هم‌زمان، peripheral یک سیگنال کلاک (SCLK) به همراه داده تولید می‌کند که امکان نرخ انتقال بالاتر و مطمئن‌تر را فراهم می‌آورد.

UART فقط ارتباط ناهم‌زمان را پشتیبانی می‌کند. USART هر دو حالت ناهم‌زمان و هم‌زمان

^۱ Universal Asynchronous Receiver-Transmitter

^۲ asynchronous

^۳ baud rate

^۴ Universal Synchronous/Asynchronous Receiver-Transmitter

را پشتیبانی می‌کند. در ثابت‌افزار مورد بررسی، peripheral‌ها منحصراً در حالت ناهم‌زمان (UART) پیکربندی و استفاده شده‌اند. بنابراین، در این بافت، اصطلاحات UART و USART اغلب به‌جای یکدیگر به‌کار می‌روند، هرچند که سخت‌افزار از نظر فنی USART است.

ب-۳ Data Register و UART Status Register

peripheral‌های USART دارای نگاشت حافظه هستند. با شروع از یک آدرس پایه (مثلاً 0x40013800 برای USART1)، ثابت‌ها در افست‌های ثابتی قرار دارند.

ب-۳-۱ ثابت وضعیت (SR) - offset +0x00

ثبات وضعیت یک ثابت فقط‌خواندنی است که بیت‌های منفرد آن وضعیت فعلی peripheral را نشان می‌دهند. ثابت‌افزار پرچم‌های خاصی را بررسی می‌کند تا علت وقوع وقفه را تشخیص دهد. بیت‌های کلیدی شناسایی‌شده در تحلیل:

- Bit 6 (TC - Transmission Complete): نشان می‌دهد که داده‌های نوشته‌شده در بافر فرستنده، به‌طور کامل روی خط TX شیفت داده شده‌اند. ISR این بیت را بررسی می‌کند تا بداند چه زمانی می‌تواند بایت بعدی را برای ارسال بارگذاری کند.
- Bit 4 (IDLE - Line Idle): نشان می‌دهد که خط RX حداقل به مدت یک فریم کامل بیکار بوده است. ISR این بیت را بررسی می‌کند تا پایان یک بسته ورودی را تشخیص داده و یک تعویض زمینه^۵ برای پردازش داده دریافتی راه‌اندازی کند.

ب-۳-۲ ثابت داده (DR) - offset +0x04

ثبات داده یک بافر خواندنی/نوشتنی است که به‌عنوان رابط بین CPU و ثابت‌های شیفت فرستنده/گیرنده عمل می‌کند.

- عمل نوشتن: CPU یک بایت در DR می‌نویسد. سخت‌افزار USART بلافاصله این بایت را به ثابت شیفت فرستنده منتقل کرده و شروع به ارسال بیت‌به‌بیت آن روی پایه TX می‌کند.

^۵ context switch

- عمل خواندن: CPU یک بایت از DR می‌خواند. این کار آخرین بایت کاملاً دریافت‌شده از ثبات شیفت گیرنده را بازیابی می‌کند.

رفتار حیاتی: انجام خواندن یا نوشتن روی DR به‌طور خودکار پرچم‌های وضعیت مرتبط (مانند پرچم‌های TC یا IDLE) را در SR پاک می‌کند. به همین دلیل است که کد اسمبلی دستور "ldr r1, [r4, #0x4]" را اجرا می‌کند، حتی زمانی که مقدار بازیابی‌شده استفاده نمی‌شود. این کار به‌منظور تأیید وقفه و بازنشانی پرچم‌های سخت‌افزاری انجام می‌شود.

ب-۴ تحلیل مقایسه‌ای USART1، USART2 و USART3

اگرچه USART1، 2 و 3 هسته عملکردی یکسانی دارند (همان قالب‌بندی، تولید نرخ باود و منطق پروتکل)، اما در پیاده‌سازی فیزیکی، اتصال به گذرگاه و تخصیص پایه‌ها روی STM32F103 با یکدیگر تفاوت دارند.

ب-۴-۱ گذرگاه فیزیکی و کارایی

peripheral‌ها به گذرگاه‌های پیشرفته peripheral^۶ متفاوتی متصل هستند:

- USART1 به گذرگاه APB2 متصل است که با فرکانس کامل سیستم کلاک (در این ثابت‌افزار 72 MHz) کار می‌کند.
- USART2 و USART3 به گذرگاه APB1 متصل هستند که حداکثر با فرکانس 36 MHz کار می‌کند.

این موضوع مستقیماً بر حداکثر نرخ باود قابل‌دستیابی تأثیر می‌گذارد. USART1 می‌تواند نرخ‌های بیتی بالاتری (تا حدود 4.5 Mbps) را نسبت به USART2/3 (حدود 2.25 Mbps) پشتیبانی کند.

ب-۴-۲ نگاشت پایه پیش‌فرض (بدون بازنگاشت)

پایه‌های I/O پیش‌فرض در جدول زیر خلاصه شده‌اند.

^۶ APB - Advanced Peripheral Bus

USART3	USART2	USART1	ویژگی
APB1 (36 MHz)	APB1 (36 MHz)	APB2 (72 MHz)	گذرگاه
PB10	PA2	PA9	پایه TX (پیش فرض)
PB11	PA3	PA10	پایه RX (پیش فرض)
APB1ENR[18]	APB1ENR[17]	APB2ENR[14]	بیت فعال سازی RCC
39	38	37	شماره IRQ
0x40004800	0x40004400	0x40013800	آدرس پایه